Quantum Computing
Algorithms and Applications

Homework 1

Due by April 16, 2026 (Thursday)

Note: You have to include the code you used to solve the problems in your assignment.

Important: Please use your **student ID number** as the random seed in all problems. For example: `seed = 0` (replace `0` with the numerical part of your student ID). This ensures that each student's results are unique and reproducible. **Failure to set the random seed will result in a penalty of 50% for each of Problems 1 to 3.**

Problem 1

Ref. [1] has shown that data reuploading circuit can be naturally interpretable as partial fourier series. Please read Ref. [1] for the details and read Ref. [2] for the code implementation. In Ref. [1], they have also discussed the case of fitting a multivariate function. Let's consider a simple case of fitting a bivariate function. Given a target function:

$$f(x_1, x_2) = \sin(e^{x_1} + x_2). \tag{1}$$

Try to train a quantum machine learning model based on data reuploading circuit with $x_1, x_2 \in [0, 0.5]$, and test on $x_1, x_2 \in [0.5, 1]$.

Here is sample code to generate the training data:

```
import numpy as np
import torch

seed = 0 # replace with your student ID
np.random.seed(seed)
torch.manual_seed(seed)

f = lambda x: torch.sin(torch.exp(x[:,0])+ x[:,1])

train_input = torch.zeros(1000, 2) # 1000 samples, 2 variables
train_ranges = np.array([0, 0.5] * 2).reshape(2, 2)

test_input = torch.zeros(1000, 2)
test_ranges = np.array([0.5, 1] * 2).reshape(2, 2)

for i in range(2):
    train_input[:, i] = (
        torch.rand(
            1000,
        )
        * (train_ranges[i, 1] - train_ranges[i, 0])
        + train_ranges[i, 0]
    )
    test_input[:, i] = (
        torch.rand(
            1000,
        )
        * (test_ranges[i, 1] - test_ranges[i, 0])
        + test_ranges[i, 0]
    )
train_label = f(train_input)
test_label = f(test_input)
```

Evaluate the model with mean squared error (MSE). Try to find the best hyperparameters of the model, such as the number of layers or the number of qubits, to make the test loss lower than 0.1.

Please include the following in your report:

(a) Plot the training loss and test loss versus epoch for your best model configuration.

(b) Provide a comparison table of at least 4 different hyperparameter configurations (e.g., number of layers, number of qubits, encoding gate type), reporting train MSE, test MSE, and the number of trainable parameters for each.

(c) Compute and plot the Fourier spectrum of both the target function and your trained model's output over the test domain. Discuss which frequency components your model captures and how this relates to the circuit depth, referencing the analysis in Ref. [1].

Problem 2

In the class we have discussed about explicit quantum model, implicit quantum kernel method and data reuploading circuit. There is also a paper to discuss the performance of these three methods on regression tasks [3]. Now, let's consider classification tasks. Follow Ref. [4], which has already implemented data reuploading circuit, try to implement explicit quantum model and implicit quantum kernel method (see Appendix C for a brief overview) and compare the performance of these three methods on two datasets:

- The **circle dataset** (as used in Ref. [4]).
- The **moons dataset** from scikit-learn (`sklearn.datasets.make_moons` with `noise=0.1`, `n_samples=200`).

Please include the following in your report:

(a) Try to obtain Fig. 6 in Ref. [3] with the performance on the circle dataset.

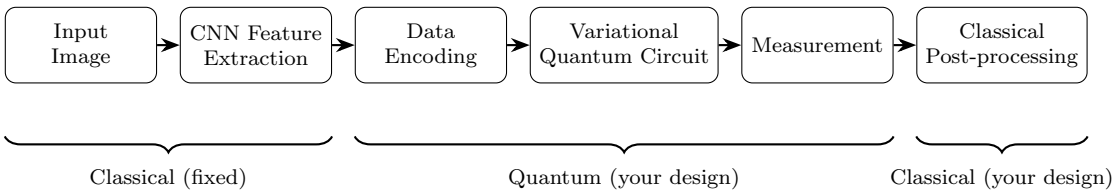
(b) Plot the decision boundaries of all three methods on both datasets (6 plots total, arranged as a 3×2 grid).

(c) Provide a comparison table reporting, for each method on each dataset: test accuracy, number of trainable parameters (or kernel evaluations), and training time.

(d) Write a brief discussion (3–5 sentences) comparing the three methods across both datasets. Relate your findings to the arguments in Ref. [3].

Problem 3

Convolutional neural networks (CNNs) are widely used in image classification tasks. Usually, a CNN extracts spatial features and passes them to a fully connected network, especially a multi-layer perceptron (MLP) as representation learning head, for classification. An alternative approach is to replace the MLP head with a parameterized quantum circuit (PQC), creating a hybrid classical-quantum model [5, 6, 7]. The overall pipeline of such a hybrid model is illustrated below:



For code reference, see the PennyLane quantum transfer learning tutorial in Ref. [8].

In this problem, you are given a **fixed CNN backbone** (shown below) that extracts a 256-dimensional feature vector from CIFAR-10 images. Your task is to design your own **quantum-enhanced neural network** to replace the MLP classification head. Specifically, you need to:

- Design a **data encoding** strategy to map the CNN feature vector (or a reduced version of it) onto qubits (e.g., angle encoding, amplitude encoding, or data reuploading).
- Design a **variational quantum circuit** with trainable parameters that processes the encoded features.

- Design a **measurement and classical post-processing** scheme that maps quantum measurement results to 10-class output probabilities.

You may use any quantum approach covered in class — explicit quantum model, implicit quantum kernel method, or data reuploading circuit — or develop a **novel quantum architecture** of your own design. For example, Ref. [9] proposes using single-qubit data reuploading circuits as quantum variational activation functions inside Kolmogorov-Arnold Networks (QKAN). You are encouraged to explore creative designs.

Hints:

- Since the CNN outputs a 256-dimensional vector but typical quantum circuits use a small number of qubits, you will likely need a classical linear layer to reduce the feature dimension before encoding.
- For 10-class classification, you can measure expectation values of multiple qubits and map them to class logits, or use other strategies.
- The CNN backbone parameters can be trained jointly with the quantum circuit, or you may freeze the CNN and only train the quantum part.

Here is the sample code for data preprocessing and the fixed CNN backbone:

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
trainset = torchvision.datasets.CIFAR10(
    root="./data", train=True, download=True, transform=transform
)
testset = torchvision.datasets.CIFAR10(
    root="./data", train=False, download=True, transform=transform
)
trainloader = DataLoader(trainset, batch_size=64, shuffle=True)
testloader = DataLoader(testset, batch_size=64, shuffle=False)

# Fixed CNN backbone -- DO NOT MODIFY
class CNNBackbone(nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn1 = nn.Conv2d(3, 32, kernel_size=3)
        self.cnn2 = nn.Conv2d(32, 64, kernel_size=3)
        self.cnn3 = nn.Conv2d(64, 64, kernel_size=3)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.pool(self.relu(self.cnn1(x)))
        x = self.pool(self.relu(self.cnn2(x)))
        x = self.pool(self.relu(self.cnn3(x)))
        x = x.view(x.size(0), -1)  # flatten: 64 * 2 * 2 = 256
        return x

# Baseline: CNN + MLP (for comparison)
class CNN_MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = CNNBackbone()
        self.classifier = nn.Linear(256, 10)

    def forward(self, x):
        x = self.backbone(x)
        return self.classifier(x)

# TODO: Design your quantum-enhanced classifier
class CNN_QNN(nn.Module):
```

```python
    def __init__(self):
        super().__init__()
        self.backbone = CNNBackbone()
        self.quantum_classifier = ...  # YOUR DESIGN HERE

    def forward(self, x):
        x = self.backbone(x)
        return self.quantum_classifier(x)
```

Use cross-entropy as the loss function and evaluate the model with accuracy. You may subsample the training data if needed for computational efficiency. For how to integrate a quantum circuit with PyTorch for end-to-end training, see Appendix B. For GPU setup on Google Colab, see Appendix A. For practical guidance on batch processing and training efficiency, see Appendix D.

Please include the following in your report:

(a) Describe your quantum circuit architecture: number of qubits, number of layers, encoding scheme, entangling structure, and measurement strategy for 10-class output.

(b) Report the CNN+MLP baseline accuracy and your CNN+QNN model accuracy.

(c) Provide a comparison table of CNN+MLP vs. CNN+QNN: test accuracy, number of trainable parameters, and training time.

(d) Plot the training curves (loss and accuracy vs. epoch) for both models.

(e) Write a brief discussion (3–5 sentences): does the quantum component help or hurt compared to the MLP? What design choices matter most?

The CNN+QNN model should achieve at least 40% test accuracy (random baseline is 10%). Higher accuracy and thoughtful analysis of your quantum architecture design will receive additional credit.

References

[1] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3), March 2021.

[2] Maria Schuld and Johannes Jakob Meyer. Quantum models as fourier series. https://pennylane.ai/qml/demos/tutorial_expressivity_fourier_series, 2020.

[3] Sofiene Jerbi, Lukas J. Fiderer, Hendrik Poulsen Nautrup, Jonas M. Kübler, Hans J. Briegel, and Vedran Dunjko. Quantum machine learning beyond kernel methods. *Nature Communications*, 14(1), January 2023.

[4] Shahnawaz Ahmed. Data-reuploading classifier. https://pennylane.ai/qml/demos/tutorial_data_reuploading_classifier, 2020.

[5] Maxwell Henderson, Samriddhi Shakya, Shashindra Pradhan, and Tristan Cook. Quanvolutional neural networks: powering image recognition with quantum circuits. *Quantum Machine Intelligence*, 2(1):2, 2020.

[6] Andrea Mari, Thomas R. Bromley, Josh Izaac, Maria Schuld, and Nathan Killoran. Transfer learning in hybrid classical-quantum neural networks. *Quantum*, 4:340, 2020.

[7] Arsenii Senokosov, Alexandr Sedykh, Asel Sagingalieva, Andrii Kurkin, and Alexey Melnikov. Quantum machine learning for image classification. *Machine Learning: Science and Technology*, 5(1):015040, 2024.

[8] Andrea Mari. Quantum transfer learning. https://pennylane.ai/qml/demos/tutorial_quantum_transfer_learning, 2019.

[9] Jiun-Cheng Jiang, Morris Yu-Chao Huang, Tianlong Chen, and Hsi-Sheng Goan. Quantum variational activation functions empower Kolmogorov-Arnold networks. *arXiv preprint arXiv:2509.14026*, 2025.

A Using Google Colab with GPU

If you do not have access to a local GPU, you can use Google Colab to run your code with a free GPU runtime.

1. Go to https://colab.research.google.com and sign in with your Google account.
2. Create a new notebook: **File** → **New notebook**.
3. Enable GPU: **Runtime** → **Change runtime type** → select **T4 GPU** (or any available GPU) → **Save**.
4. Install required packages by running the following in a code cell:

```
!pip install pennylane torch torchvision
# or for Qiskit:
!pip install qiskit qiskit-machine-learning
# or for CUDA-Q:
!pip install cuda-quantum
```

5. Verify GPU is available:

```
import torch
print(torch.cuda.is_available())  # should print True
print(torch.cuda.get_device_name(0))  # e.g., Tesla T4
```

6. To use the GPU in PyTorch, move your model and data to the GPU:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
# for each batch:
inputs, labels = inputs.to(device), labels.to(device)
```

Tips:

- Colab sessions time out after inactivity. Save your work frequently.
- PennyLane's `default.qubit` and Qiskit's `AerSimulator` run on CPU. GPU acceleration mainly benefits the classical CNN training in Problem 3. For quantum simulation with GPU, you can use PennyLane's `lightning.gpu` or CUDA-Q.

B Quantum Framework Quick Examples

Below are minimal examples showing how to create a parameterized quantum circuit and integrate it with PyTorch for gradient-based training. You may use any of these frameworks (or others) for your solutions.

PennyLane + PyTorch

```
import pennylane as qml
import torch
import torch.nn as nn

dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev, interface="torch", diff_method="backprop")
def circuit(params, x):
    qml.RX(x[0], wires=0)
    qml.RX(x[1], wires=1)
    qml.RY(params[0], wires=0)
    qml.RY(params[1], wires=1)
    qml.CNOT(wires=[0, 1])
    return qml.expval(qml.PauliZ(0))

# wrap as a PyTorch layer
class QuantumLayer(nn.Module):
    def __init__(self):
```

```
        super().__init__()
        self.params = nn.Parameter(torch.randn(2))

    def forward(self, x):
        return circuit(self.params, x)

model = QuantumLayer()
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)

# training loop example
x = torch.tensor([0.3, 0.5])
target = torch.tensor(0.8)
for epoch in range(50):
    optimizer.zero_grad()
    loss = (model(x) - target) ** 2
    loss.backward()  # gradients flow through the quantum circuit
    optimizer.step()
print(f"Final loss: {loss.item():.6f}")
```

Qiskit + PyTorch

```
from qiskit.circuit import QuantumCircuit, Parameter
from qiskit.primitives import StatevectorEstimator
from qiskit.quantum_info import SparsePauliOp
from qiskit_machine_learning.connectors import TorchConnector
from qiskit_machine_learning.neural_networks import EstimatorQNN
import torch

theta = [Parameter(f"t{i}") for i in range(2)]
inp = [Parameter(f"x{i}") for i in range(2)]
qc = QuantumCircuit(2)
qc.rx(inp[0], 0)     # data encoding
qc.rx(inp[1], 1)
qc.ry(theta[0], 0)   # trainable
qc.ry(theta[1], 1)
qc.cx(0, 1)

estimator = StatevectorEstimator()
qnn = EstimatorQNN(
    circuit=qc, input_params=inp, weight_params=theta,
    observables=[SparsePauliOp("ZI")], estimator=estimator,
)
model = TorchConnector(qnn)  # wraps as nn.Module

# now use model in standard PyTorch training
x = torch.tensor([[0.3, 0.5]])
output = model(x)  # differentiable, supports loss.backward()
print(output)
```

CUDA-Q + PyTorch

```
import cudaq
import torch
import torch.nn as nn
import numpy as np

@cudaq.kernel
def circuit(params: list[float], x: list[float]):
    q = cudaq.qvector(2)
    rx(x[0], q[0])
    rx(x[1], q[1])
    ry(params[0], q[0])
    ry(params[1], q[1])
    cx(q[0], q[1])

class CudaQLayer(torch.autograd.Function):
    @staticmethod
    def forward(ctx, params, x):
        ctx.save_for_backward(params, x)
        p, xi = params.detach().numpy().tolist(), x.detach().numpy().tolist()
```

```
        result = cudaq.observe(circuit, cudaq.spin.z(0), p, xi)
        return torch.tensor(result.expectation(), dtype=params.dtype)

    @staticmethod
    def backward(ctx, grad_output):
        params, x = ctx.saved_tensors
        shift = np.pi / 2
        grads = []
        for i in range(len(params)):
            p_plus = params.detach().clone(); p_plus[i] += shift
            p_minus = params.detach().clone(); p_minus[i] -= shift
            xi = x.detach().numpy().tolist()
            r_plus = cudaq.observe(circuit, cudaq.spin.z(0),
                                   p_plus.numpy().tolist(), xi)
            r_minus = cudaq.observe(circuit, cudaq.spin.z(0),
                                    p_minus.numpy().tolist(), xi)
            grads.append((r_plus.expectation() - r_minus.expectation()) / 2)
        return grad_output * torch.tensor(grads, dtype=params.dtype), None

# usage in PyTorch
params = torch.randn(2, requires_grad=True)
x = torch.tensor([0.3, 0.5])
output = CudaQLayer.apply(params, x)
loss = (output - 0.8) ** 2
loss.backward()  # parameter-shift gradients
print(f"Gradients: {params.grad}")
```

C Overview of QML Methods

This appendix provides a brief overview of the three quantum machine learning approaches referenced in Problem 2. For a detailed theoretical comparison, see Ref. [3].

Explicit Quantum Model

An explicit quantum model encodes classical input $\boldsymbol{x}$ into a quantum state via an encoding circuit $S(\boldsymbol{x})$, applies a trainable parameterized quantum circuit $W(\boldsymbol{\theta})$, and measures an observable $\hat{M}$:

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \langle 0|S^\dagger(\boldsymbol{x})\, W^\dagger(\boldsymbol{\theta})\, \hat{M}\, W(\boldsymbol{\theta})\, S(\boldsymbol{x})|0\rangle. \tag{2}$$

The parameters $\boldsymbol{\theta}$ are trained via gradient descent (using the parameter-shift rule or backpropagation through the simulator).

Implicit Quantum Kernel Method

The implicit quantum kernel method uses a *fixed* encoding circuit $S(\boldsymbol{x})$ with no trainable quantum parameters to define a kernel:

$$k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \left|\langle 0|S^\dagger(\boldsymbol{x}_i)\, S(\boldsymbol{x}_j)|0\rangle\right|^2. \tag{3}$$

The kernel matrix $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$ is computed using a quantum circuit and then passed to a classical support vector machine:

```
from sklearn.svm import SVC
K_train = compute_kernel_matrix(X_train)  # using quantum circuit
svc = SVC(kernel="precomputed")
svc.fit(K_train, y_train)

K_test = compute_kernel_matrix(X_test, X_train)
predictions = svc.predict(K_test)
```

The expressiveness comes entirely from the choice of encoding circuit $S(\boldsymbol{x})$, not from trainable quantum parameters.

Data Reuploading Circuit

A data reuploading circuit interleaves L layers of data encoding and trainable gates:

$$U(\boldsymbol{x};\boldsymbol{\theta}) = \prod_{l=1}^{L} W(\boldsymbol{\theta}_l)\, S(\boldsymbol{x}). \tag{4}$$

The input $\boldsymbol{x}$ is re-encoded in every layer, allowing the model to express higher-frequency Fourier components. As shown in Ref. [1], the number of layers L directly controls the accessible frequency spectrum. See Ref. [4] for a complete implementation example.

D Practical Tips

Batch Processing with Quantum Circuits

The examples in Appendix B process one input at a time. For mini-batch training, loop over samples in the batch:

```
class HybridModel(nn.Module):
    def forward(self, x):
        # x: (batch_size, n_features)
        out = torch.stack([self.quantum_circuit(x_i) for x_i in x])
        return out
```

In PennyLane with `interface="torch"`, you can also pass batched inputs directly to a QNode for improved performance (parameter broadcasting).

Computing Fourier Spectrum (Problem 1)

To analyze the frequency content of a 2D function, evaluate it on a regular grid and compute the 2D discrete Fourier transform:

```
import numpy as np
# evaluate on a grid over the test domain
x1 = np.linspace(0.5, 1.0, 64)
x2 = np.linspace(0.5, 1.0, 64)
X1, X2 = np.meshgrid(x1, x2)
values = f(X1, X2)  # or model predictions

# 2D FFT and magnitude spectrum
spectrum = np.fft.fft2(values)
magnitude = np.abs(np.fft.fftshift(spectrum))
```

Plot the magnitude spectrum using `plt.imshow` or `plt.contourf`. Compare which frequency peaks appear in the target function vs. your trained model.

Common Data Encoding Strategies

Several strategies exist for encoding classical data $\boldsymbol{x} \in \mathbb{R}^n$ into a quantum circuit:

- **Angle encoding**: Apply single-qubit rotations $R_Y(x_i)$ or $R_X(x_i)$ to each qubit. Requires n qubits for n features.
- **Amplitude encoding**: Encode the data vector into the amplitudes of a quantum state, requiring only $\lceil \log_2 n \rceil$ qubits. More qubit-efficient but the state preparation circuit is more complex.
- **Data reuploading**: Re-encode the data in each circuit layer (see Appendix C). Increases expressiveness without adding qubits.

For Problem 3, where the CNN outputs a 256-dimensional vector, you will typically need a classical layer (e.g., `nn.Linear(256, n_qubits)`) to reduce the dimension before quantum encoding.

Training Efficiency

Quantum circuit simulation is significantly slower than classical neural network computation. To manage training time:

- Start with small circuits (2–4 qubits, 1–2 layers) and scale up once your pipeline works.
- For Problem 3, subsample the training data during development (e.g., 5000–10000 samples) and use the full dataset for your final run.
- GPU acceleration mainly benefits the CNN backbone. Most quantum simulators (e.g., `default.qubit`) run on CPU. Plan your training time accordingly.